Enrolment id: 202511043

Name: Umang Limbasiya

Iterative AI-Assisted Requirements Engineering System with Manual Quality Evaluation

1. Introduction

This document describes the design and implementation of an Iterative AI-Assisted Requirements Engineering System that converts spoken system descriptions into structured Functional and Non-Functional Requirements (FRs/NFRs), enables iterative refinement through clarification questions, and evaluates requirement quality using established standards including IEEE 830-1998, ISO/IEC/IEEE 29148:2018, Saito et al. (3Cs), Saavedra et al. (quality properties), and Stephens (requirements quality metrics).

The system integrates automatic requirement generation with human-in-the-loop refinement and structured manual evaluation.

---

2. System Architecture

The system follows a modular architecture consisting of:

1. Speech-to-Text Module
2. Requirement Generation Module
3. Gap Analysis Module
4. Iterative Refinement Module
5. Manual Evaluation Framework

2.1 Technology Stack

- Frontend + Backend: Streamlit (single Python application)
- Speech Recognition: OpenAI Whisper (base model)
- Large Language Model: LLaMA 3.2 via Ollama (local inference)
- Programming Language: Python
- Evaluation Tool: Microsoft Excel (manual scoring)

---

3. Implementation Details

3.1 Speech-to-Text Processing

The system uses the Whisper base model to transcribe uploaded audio files. The model is loaded once using Streamlit's @st.cache_resource to avoid repeated loading overhead.

Implementation steps:

1. User uploads audio (WAV, MP3, WEBM, M4A).

2. Audio is stored temporarily using Python's tempfile.
3. Whisper transcribes the audio into text.
4. Temporary file is removed.

This produces a transcript that becomes the primary input for requirement extraction.

3.2 Initial Requirement Generation

The transcript is passed to the LLM (LLaMA 3.2) with a strictly structured prompt enforcing:

- Separation of Functional and Non-Functional Requirements
- Format: "The system shall …"
- JSON-only output
- No explanatory text

To prevent parsing failures caused by extra model-generated text, a robust JSON extraction function was implemented:

- The system locates the first "{" and last "}"
- Extracts the enclosed JSON
- Parses using json.loads()

This ensures stability even if the model produces introductory sentences.

3.3 Iterative Gap Analysis

After initial generation, the system allows iterative improvement through a "Gap Analysis" button.

When triggered:

- The current requirements are sent to the LLM.
- The prompt strictly enforces:
    - Only questions
    - One per line
    - No numbering
    - No headings
    - No explanations

Each generated question is dynamically displayed with its own input field using Streamlit session state. This ensures:

- One answer per question
- Controlled refinement

- Structured clarification input

3.4 Requirement Improvement Loop

User answers are combined with the existing requirement set and sent back to the LLM for refinement.

The improvement prompt enforces:

- No hallucinated features
- Retention of valid requirements
- Strict JSON output
- No additional commentary

After improvement:

- Requirements are updated
- Version counter increments
- Question state resets
- The loop can repeat until the user selects “Verified”

This creates a human-in-the-loop iterative refinement system.

---

4. Gold Baseline Construction

To enable objective evaluation, a Gold Baseline Requirement Set was manually derived from the transcript.

The baseline:

- Separates atomic requirements
- Expands implicit logical necessities (e.g., authentication enforcement)
- Distinguishes encryption at rest and in transit
- Splits mobile and desktop compatibility

Baseline totals:

- 7 Functional Requirements
- 5 Non-Functional Requirements

This baseline remains fixed and is used for all comparative evaluations.

---

5. Evaluation Methodology

Evaluation is performed manually in Excel using three core metrics:

1. Completeness
2. Correctness
3. Redundancy

Separate scores are computed for FR and NFR.

5.2 Correctness

Valid requirements:

- Match baseline intent
- Contain no hallucinations
- Introduce no contradictions
- Avoid ambiguity

6. Experimental Findings

Across three iterations:

Iteration	Overall Score
1	8.86
2	7.94
3	7.36

Observed behavior:

- Completeness remained constant.
- Correctness decreased due to:
    - Over-specification (Google OAuth).
    - Unsupported assumptions (partial editing).
    - Hallucinated accessibility constraints.
- Redundancy remained stable.

This demonstrates:

Iterative LLM refinement does not guarantee improvement and may introduce semantic drift or hallucinations without strong grounding constraints.

---

7. Alignment with Standards

The evaluation aligns with:

- IEEE 830: clarity, completeness, verifiability.

- ISO/IEC/IEEE 29148: correctness and consistency.
- Saito et al.: 3Cs (Completeness, Consistency, Correctness).
- Saavedra et al.: atomicity and redundancy.
- Stephens: quantitative requirement scoring.

---

8. Conclusion

This system demonstrates a hybrid AI-assisted requirements engineering workflow combining:

- Automated extraction
- Human clarification
- Structured iteration
- Quantitative evaluation

The experiment reveals that while AI assists in rapid requirement drafting, manual validation and structured evaluation remain essential for maintaining correctness and preventing hallucination-driven degradation.

The framework provides a reproducible method to measure requirement quality evolution across iterations and supports empirical study of LLM-assisted requirements engineering.